

Language-Guided Semantic Grasp and Place on a Sagittarius Robot Arm

Team Number: 3

Junlin Xiao (122090599) – System Integration, Task Planning, Perception & Execution

Xiaohang Li (123090291) – Perception Pipeline, Motion Planning & Implementation

Yulu He (224040211) – Experimental Support

Zhihan Yu (225040301) – Documentation Support

Repository: <https://github.com/Block-02/AIR-5051-2026--Team3-FinalProject>

1 Project Overview

This project upgrades the original Sagittarius color-sorting demo into a language-guided semantic manipulation system. Instead of recognizing only pre-coded color blobs, the robot accepts natural-language instructions, detects semantically matched targets in the camera image, maps the image coordinates to robot-plane coordinates, and executes grasp or placement with the existing Sagittarius motion stack.

The project goal is to make the robot behave less like a fixed demo and more like a small embodied agent. A user can issue a short object query such as **red block**, an object-to-target command such as **put blue block into red bucket**, or a task-level instruction such as **put each block into the bucket of the same color**. The system then parses the request, decides which object should be picked first, detects the relevant target, and executes a grasp/place pipeline.

The core intelligence comes from combining a vision-language detector with geometry-aware robot execution. We use GroundingDINO as the semantic detector. GroundingDINO is an open-set object detector that takes a text prompt and an image, then returns bounding boxes and confidence scores for regions that semantically match the prompt. This makes it suitable for language-guided robotics because the user does not have to select from a rigid list of class IDs. However, detector boxes alone are not accurate enough for real grasping. For that reason, the project separates semantic detection from geometric center estimation: GroundingDINO chooses *what* to act on, and an HSV contour-based refinement step estimates *where* to grasp or place more precisely inside the selected region.

The hardware setup uses a Sagittarius robot arm, a USB camera, ROS Noetic, MoveIt, and the existing `sgr_ctrl` action interface. In our final demo-oriented setup, the main runtime path uses a front-view camera pose with CUDA-enabled GroundingDINO inference on the development machine. This setup was chosen because it produced the most stable integration result during real-robot testing.

2 Design & Implementation

2.1 GroundingDINO in the Project

GroundingDINO is used here as a language-conditioned perception front-end rather than as a complete manipulation solution. Given a text phrase such as **blue block**, **red bucket**, or

`palm`, it proposes image regions that match the phrase. This open-vocabulary behavior is the main reason we selected it over a rigid closed-set detector. It allows the robot to interpret commands at the language level and keeps the project closer to the idea of semantic robotics instead of simple color-threshold scripting.

At the same time, the project deliberately avoids over-trusting the detector output. A bounding box center is often good enough to show that the model understood the object category, but it is usually not precise enough for manipulation. The grasp point for a small block and the placement center for a bucket opening both require better geometric localization than the raw box center can reliably provide. This gap between semantic correctness and physical precision strongly influenced our design.

2.2 Pipeline Overview

Figure 1 summarizes the runtime pipeline. The high-level flow is:

1. receive a language command;
2. parse it into one or more pick/place subtasks;
3. run GroundingDINO to locate the current semantic target;
4. refine the target center inside the selected ROI when color-based refinement is available;
5. verify confidence and multi-frame stability;
6. map the image center to robot-plane coordinates through calibration;
7. execute grasp or placement with the existing Sagittarius stack.

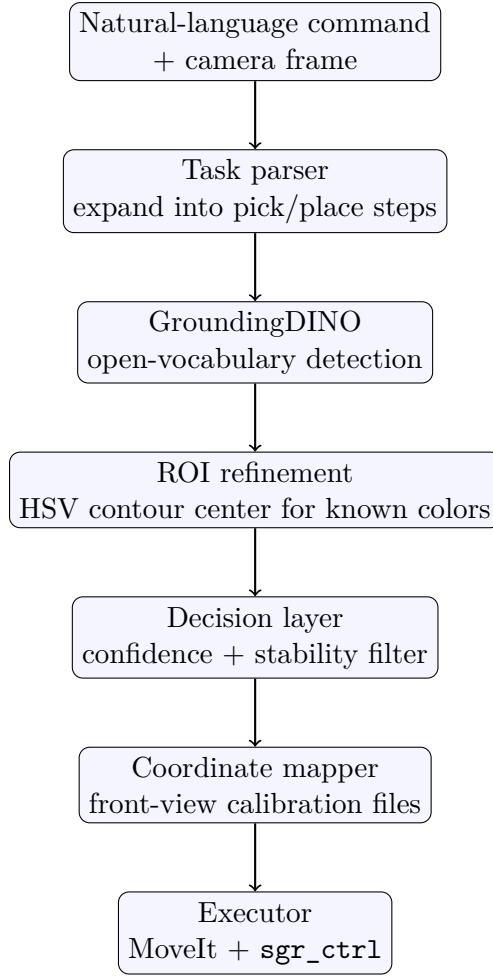


Figure 1: Semantic manipulation pipeline in the current Sagittarius system.

2.3 Task Planning: the “Brain”

The task-planning layer translates natural-language instructions into executable steps. For example, the instruction `put each block into the bucket of the same color` is expanded into:

- `red block -> red bucket`
- `blue block -> blue bucket`
- `green block -> green bucket`

This decomposition matters because the robot does not solve the whole task in one shot. Instead, it iterates through a state machine: observe, detect, lock the target over multiple stable frames, grasp, re-observe, detect the placement target, and place. The same structure also supports simpler commands such as `put blue block into red bucket` and `put in the palm`.

2.4 Robot Skills: the “Actions”

The runtime is built from a set of reusable robot skills:

- **Semantic target selection:** choose the image region that best matches the text query.
- **Center refinement:** recover a better geometric center from the selected ROI.
- **Coordinate mapping:** convert pixel coordinates into workspace coordinates.

- **Grasp:** call the existing Sagittarius grasp action.
- **Place:** move to a placement target using a dedicated placement calibration.

This decomposition was intentionally chosen so that the project remains understandable and extensible at the engineering level. Each stage can be improved without rewriting the whole pipeline.

2.5 Calibration and Coordinate Mapping

The perception stack outputs image coordinates, while the robot executes in workspace coordinates. To bridge this gap, the project uses manually collected calibration pairs and fits linear mappings from image pixels to robot-plane coordinates. The current runtime uses separate front-view calibration files for grasping and placement:

- `vision_config_pick_front.yaml`
- `vision_config_place_front.yaml`

This split is important. Picking a block and placing it into a bucket are not geometrically identical operations, even if the same camera is used. The latest runtime also uses the same center-refinement logic during calibration and during execution, which avoids a previous mismatch where the calibration target center and the runtime target center were defined differently.

2.6 Engineering Innovation

From an engineering perspective, the main innovation is not just that the robot uses GroundingDINO. The more important contribution is the modular pipeline around it.

- **Detector-agnostic design:** the perception stage is wrapped behind a backend interface. GroundingDINO is the current backend, but the architecture is intentionally prepared so that another detector, such as YOLO or a different grounding model, could replace it later.
- **Semantic-geometric decoupling:** the system does not force one model to do everything. Semantic understanding is handled by the detector, while precise target geometry is handled by a separate center-refinement layer.
- **Safety-first robotics behavior:** the robot rejects low-confidence or unstable targets instead of always trying to execute.
- **Reuse of legacy robot control:** the project extends the original Sagittarius stack instead of discarding it, which reduced integration risk and made the system more realistic to deploy.

We did not benchmark a YOLO-based backend in this project, so we do not claim comparative detector results. The point is that the software structure now supports that future substitution, which is a meaningful system-level improvement over a tightly coupled one-off demo.

3 Experiments & Results

3.1 Test Environment

Real-robot experiments were conducted on a tabletop manipulation setup with colored blocks, colored buckets, and handover-style interaction for the palm demo. The perception system used a front-facing USB camera stream at 640x480 resolution. Motion execution was handled by the physical Sagittarius arm through ROS Noetic, MoveIt, and the existing `sgr_ctrl` interface. The final demo path emphasized front-view observation because that was the most stable integrated configuration.

3.2 Successful Demo Scenarios

The final project includes three successful demo scenarios that capture different aspects of the system:

Demo	Status	What it demonstrates
Blue block $\rightarrow$ red bucket	Success	The robot can follow a cross-color language instruction rather than only matching identical categories.
Three blocks $\rightarrow$ their own colored buckets	Success	The robot can parse a higher-level command into multiple semantic subtasks and complete them sequentially.
Put in the palm	Success	The system supports a different placement target and shows the flexibility of language-guided object handover.

Table 1: Three successful demos used to present the final system.

These demos matter because they test different parts of the stack. The blue-block-to-red-bucket demo emphasizes semantic grounding across object categories. The same-color sorting demo emphasizes multi-step planning and repeated re-observation. The palm demo emphasizes that the target need not always be another tabletop object, which makes the system feel less like a hard-coded sorting script.

3.3 Pipeline Walkthrough Using the Annotated Detection Frame

Figure 2 is a recorded annotated frame from the front-view runtime. It is a useful figure for explaining how the pipeline works on a real example.



Figure 2: Annotated runtime image from the front-view semantic grasp pipeline.

This figure can be read as a compact visualization of the full pipeline:

- the text query determines what object the perception module should search for;
- GroundingDINO proposes a semantic bounding box around the target;
- the ROI inside that box is then refined to produce a more useful geometric center;
- once the center is stable, the calibration module maps it to robot coordinates;
- the execution layer sends the arm to grasp or place accordingly.

This is also the best example of why the project separates semantic selection from geometric refinement. The box is useful because it tells the robot *which* object is intended, but the final grasp or placement center must be computed more carefully than simply taking the box midpoint.

3.4 Challenges and Solutions

The biggest challenges came from the interaction between perception precision, calibration quality, and real hardware behavior.

- **Detector box centers were too coarse for manipulation.** A semantically correct bounding box is not guaranteed to align with the true center of a block or bucket. The solution was to keep GroundingDINO for semantic detection and add HSV contour center refinement inside the selected ROI.
- **One calibration was not enough for all phases.** Reusing one mapping for both pick and place reduced placement quality. The solution was to maintain separate `pick_front` and `place_front` calibration files.
- **Some apparent software failures were actually hardware issues.** During debugging, startup and motion failures were sometimes traced back to robot condition rather than perception logic. This changed our workflow: before blaming the software stack, we first verify the physical arm state.

4 Photo & Video Documentation

The report package includes the representative experiment image used in Figure 2. For the final submission, the team should add links to the recorded demo videos corresponding to the three successful cases listed in Table 1.

- Blue block into red bucket (Demo 1): [\[Video\]](#)
- Three blocks into matching buckets (Demo 2): [\[Video\]](#)
- Put in the palm (Demo 3): [\[Video\]](#)

5 Conclusion

The final system can interpret natural-language manipulation commands, detect semantically described targets with GroundingDINO, refine the target center for more reliable geometry, map the target to robot coordinates, and execute grasp/place actions with a Sagittarius arm. Compared with the original color-sorting baseline, the project now supports richer instructions and more varied targets while still preserving the practical control stack that works on the real robot.

The most important result is architectural rather than purely algorithmic. The project now has a reusable semantic manipulation pipeline in which the detector, the refinement logic, the

calibration module, and the executor are explicitly separated. That makes the system easier to debug, easier to extend, and easier to adapt to another detector backend in future work.

The next engineering improvements are clear:

- test an alternative detector backend such as YOLO within the same framework;
- expand the refinement layer beyond simple color-guided center estimation;
- improve bucket-center localization for more reliable placement;
- collect denser calibration coverage across the full workspace.